



Livello Trasporto

Parte 3

Internet, Web e Cloud
Claudia Canali

Problemi da affrontare nel TCP



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

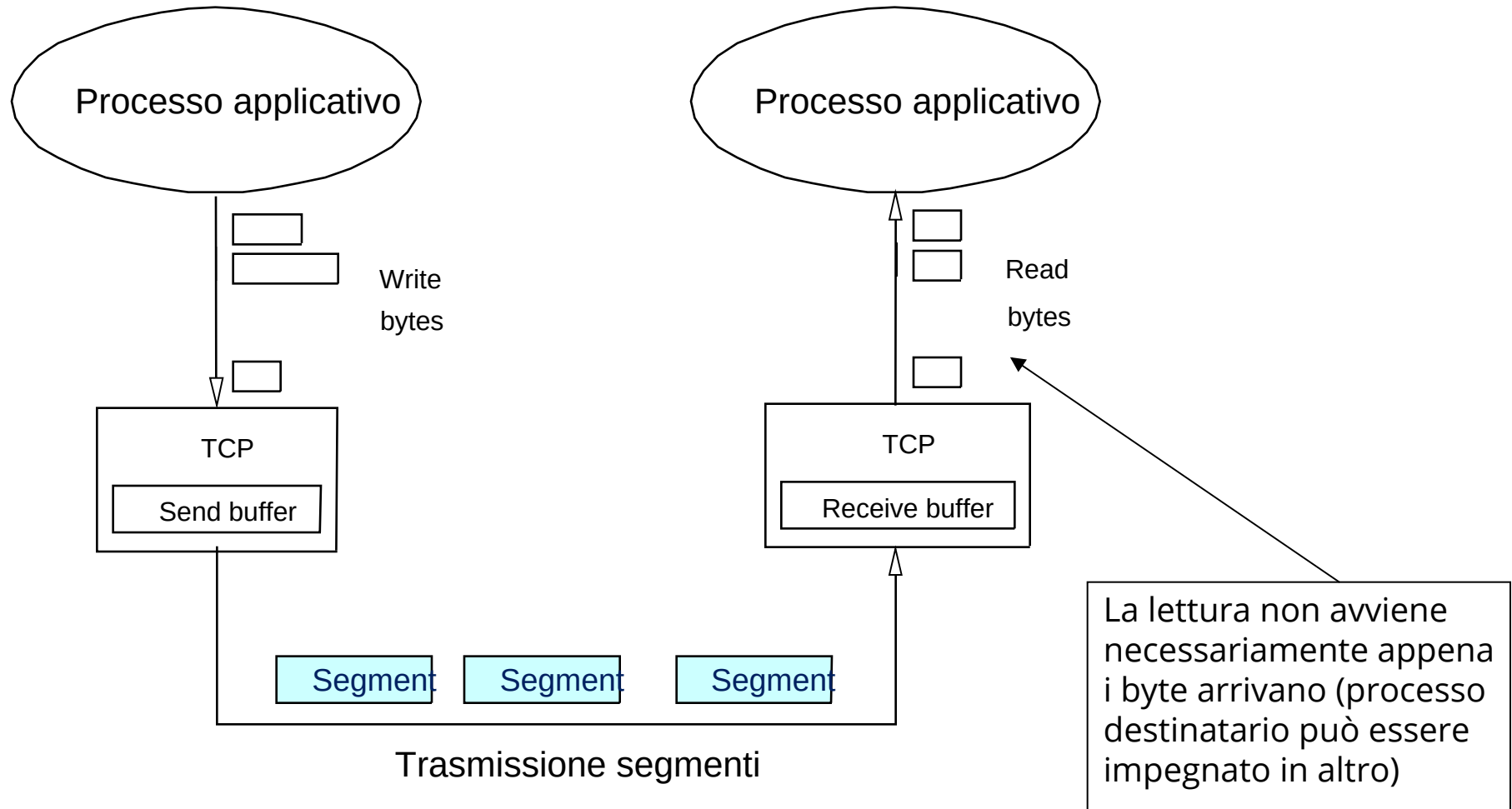
- *Il TCP è il livello che deve evitare di spedire più segmenti di quanti il destinatario sia in grado di riceverne*
 - **“Controllo di flusso”**
- *Il TCP è il livello che deve evitare di spedire più segmenti di quanti la rete tra mittente e destinatario sia in grado di assorbirne*
 - **“Controllo di congestione”**



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

TCP: Controllo di Flusso

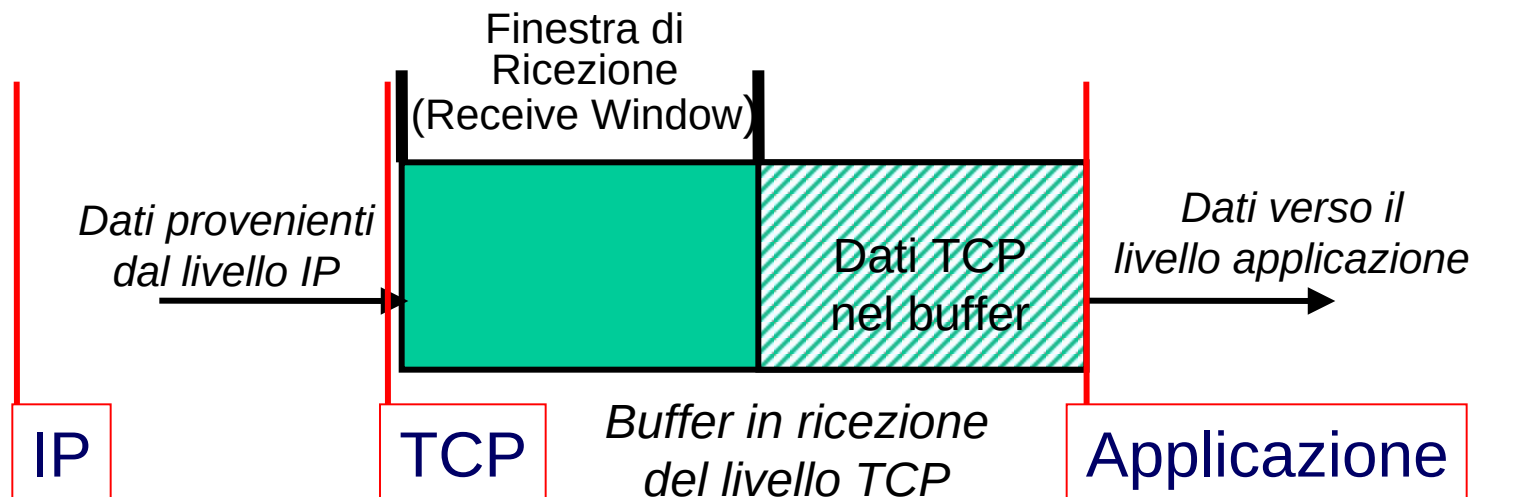
Trasmissione dati nel TCP



Controllo di flusso



- **Obiettivo:** il mittente non deve inviare una quantità di dati superiore a quella che può essere contenuta nel buffer del destinatario (**overflow del buffer**)
 - Evitare di inviare dati ad un tasso di trasmissione troppo elevato
- Il destinatario **informa esplicitamente** il mittente della quantità di **spazio libero nel buffer ricezione TCP (Receive Window – campo segmento TCP)**
- La dimensione della Receive Window **varia dinamicamente** → ogni segmento TCP ACK informa il mittente del **numero massimo di byte** ricevibili
- Comunicazione **full-duplex**: due finestre di ricezione distinte



Controllo di flusso a livello di end-point



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

- **L'ACK è inviato a livello di segmento**
 - Vale per byte multipli
- **Dall'ACK si possono dedurre due informazioni:**
 - Quanti byte sono stati ricevuti dal destinatario (in ordine)
 - Quanti byte il destinatario può ancora ricevere (*in quel momento*)
- **Il mittente regola la sua finestra (sliding window)** in base alla disponibilità che gli è stata indicata dal destinatario
 - Se il destinatario ha esaurito il buffer, indica una disponibilità pari a 0
- In tal caso, la **trasmissione si sospende** e riprende solo quando il destinatario segnala di essere nuovamente in grado di ricevere byte

Calcolo finestra di ricezione



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

- **Host A sta inviando un file di grandi dimensioni all'host B**
- B alloca un **buffer di ricezione di dimensione RcvBuffer**
- Definiamo:
 - **LastByteRead**: numero dell'ultimo byte del flusso dati che il processo applicativo ha letto dal buffer (consegnato)
 - **LastByteRcvd**: numero dell'ultimo byte che proviene dalla rete (in ordine) e che è stato copiato nel buffer di ricezione di B
- Per non mandare in overflow il buffer di ricezione dovremo avere:
 - **LastByteRcvd – LastByteRead $\leq$ RcvBuffer**
- **Finestra di ricezione rwnd** impostata a spazio disponibile:
 - **rwnd = RcvBuffer - [LastByteRcvd – LastByteRead]**
- *Come viene usata questa informazione (inserita nel segmento TCP) dal lato del mittente?*

**Disponibilità
iniziale**

Calcolo finestra di ricezione



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

- **A sua volta, l'host A (mittente) tiene traccia di due variabili**
 - **LastByteSent**: ultimo byte mandato
 - **LastByteAcked**: ultimo byte di cui si è ricevuto l'ACK
- Differenza tra i valori: quantità di dati spediti da A per cui manca ACK
 - Mantenendo tale quantità minore del valore di **rwnd**, A è sicuro di non mandare in overflow il buffer di B
- A si assicura che
 - **LastByteSent – LastByteAcked $\leq$ rwnd**per tutta la durata della connessione

Ricordiamo:

In caso il buffer di B sia pieno, A sospende la **trasmissione** e riprende solo quando il destinatario segnala di essere **nuovamente in grado** di ricevere byte

Possibile problema tecnico: quale?

Come riprendere da una window = 0?



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

- Una volta che il destinatario ha comunicato una finestra di dimensione 0
 - ☑ Il mittente non può più inviare dati
 - ☑ Il mittente non può più conoscere se la finestra di ricezione è aumentata perché il destinatario non invia messaggi spontaneamente, ma solo ACK in risposta a segmenti in arrivo

SOLUZIONE TCP: Quando il mittente riceve una Receive Window $rwnd=0$, continua ad inviare periodicamente un segmento con 1 byte di dati finché non riceve indicazione di spazio disponibile

Animazione interattiva



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

- **Animazione interattiva**

https://media.pearsoncmg.com/curriculum/intl/it/lab/9788891902559_KUROSE_ROSS/studente/interactive%20animations/flow-control/index.html

- **Host B consumes data in 2Kbyte chunks at random times.**
- **Provate**
 - **File Size: 4 KB e Buffer Size: 2 KB**
 - **File Size: 16 KB e Buffer Size: 4 KB**
- **Stoppate e osservate i valori**
- **Provate altre combinazioni**



TCP: Controllo di Congestione

Controllo di congestione



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

- **Congestione (def.):**

- Informalmente: “un numero elevato di sorgenti inviano contemporaneamente troppi dati generando un traffico che la rete (Internet) non è in grado di gestire”

- ***Diverso da Controllo di flusso!***

- **Effetti della congestione:**

- Perdita di pacchetti (overflow dei buffer nei router)
- Lunghi ritardi (tempi di attesa nei buffer dei router)

Controllo di congestione: due approcci possibili



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Controllo di congestione end-to-end

- .Il livello di rete non fornisce un supporto esplicito al livello di trasporto
- .La situazione di congestione è determinata **analizzando le perdite di pacchetti ed i ritardi nei nodi terminali**
- .*Approccio utilizzato dal TCP*

Controllo di congestione assistito dalla rete (*approcci “teorici”*)

- .I router forniscono un **feedback esplicito** al mittente riguardante lo stato di congestione nella rete
 - **Singolo bit** che indica la congestione di un link
 - **Misura della congestione nei router**: lunghezza della coda dei buffer
- .**Meccanismo di feedback più sofisticato** (es., controllo di congestione ATM ABR - available bit rate): esplicita comunicazione al mittente della frequenza trasmissiva disponibile in uscita per un router

- **CONTROLLO DI CONGESTIONE “END-TO-END”**
 - Regolato dal mittente, non dalla rete
- Limite alla velocità d’invio dei dati in base alla **congestione percepita sulla rete**
- **Rilevazione congestione sul percorso verso la destinazione**
 - Evento ‘perdita’ di un pacchetto
 - Occorrenza di un Timeout
 - Ricezione di 3 ACK duplicati dal destinatario
- Utilizzo di variabile aggiuntiva
 - **Finestra di congestione - congestion window (cwnd)**

Controllo di congestione in TCP



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

- **Ritrasmissione** → **sintomo di congestione**
- **Ma al tempo stesso la ritrasmissione** contribuisce ad aumentare la congestione della rete in atto (più pacchetti in circolo)
- → **TCP riduce il tasso di trasmissione in caso di congestione**
 - **Congestion Window (cwnd)**: dimensione della finestra di congestione
 - **Receive Window (rwnd)**: dimensione della finestra di ricezione
- La **cwnd** impone un limite alla velocità di trasmissione del mittente

$$\text{LastBytesent} - \text{LastByteAcked} = \min(\text{rwnd}, \text{cwnd})$$

Nel caso in cui il processo applicativo destinatario non sia molto lento e il buffer di ricezione sia abbastanza capiente, la velocità della trasmissione è nella pratica limitato dalla cwnd

Controllo di congestione in TCP



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Misura delle prestazioni di una connessione TCP:

- Throughput (tasso di trasmissione)

$$\text{throughput} = \frac{w * \text{MSS}}{\text{RTT}} \quad \text{Bytes/sec}$$

***w → limitato
dalla cwnd***

w = numero di segmenti nella finestra di invio

MSS = dimensione massima del segmento

RTT = Round Trip Time

Aumentare al massimo le prestazioni – approccio generale

- **idealmente**: trasmettere il più velocemente possibile senza perdita di segmenti (valore massimo di CongestionWindow)
- **incrementare** CongestionWindow il più possibile, finché non si verifica un episodio di congestione
- **diminuire** CongestionWindow, ricominciando poi a incrementarla nuovamente

Controllo di congestione in TCP



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

- **Questione critica:** come determinare il tasso di trasmissione iniziale, incrementi e decrementi? **Trade-off**
 - Se mittenti TCP troppo veloci → collasso di congestione
 - Se mittenti TCP troppo lenti → sottoutilizzo banda disponibile
- No controllo centralizzato ma approccio distribuito:
 - **TCP auto-temporizzato attraverso gli ACK**
- Utilizzo di ACK per temporizzare incrementi o decrementi della cwnd
 - ACK ricevuto correttamente: indicazione implicita funzionamento → la cwnd può essere incrementata
 - Evento perdita: indicazione implicita di congestione → cwnd deve essere decrementata

Algoritmo Controllo di Congestione TCP



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

- **Stato stazionario (non di congestione):** dimensione della finestra pari a quella indicata dal ricevente (per il controllo di flusso)
- **Stato di congestione:** riduzione della dimensione della finestra
- **Algoritmo di controllo di congestione TCP [RFC 5681]**
- Tre fasi principali
 - 1) Slow start
 - 2) Congestion Avoidance
 - 3) Fast Recovery

Slow Start



- All'inizio dell'utilizzo di una nuova connessione la dimensione della finestra di congestione **cwnd** viene **inizializzata a 1 MSS**
- **Incremento progressivo** della dimensione della finestra
 - Tentativo di scoprire velocemente la banda disponibile
 - **cwnd** incrementata di 1 MSS per ogni ACK ricevuto (non duplicato)
 - **Crescita esponenziale** – raddoppio velocità ad ogni RTT
- **Quando termina incremento esponenziale?**
 - 1) **Evento di perdita (timeout)**
 - **cwnd = 1 MSS** e riparte processo di **slow start**
 - Set di **threshold ssthresh** (*slow start threshold*) a **cwnd/2**
 - 2) **Raggiungimento di ssthresh**: quando **cwnd = ssthresh** la fase di incremento esponenziale termina e si raggiunge lo stato stazionario di incremento lineare (entra in modalità congestion avoidance)

Metà del valore di
cwnd quando si è
rilevata
congestione

←

Slow Start

Algoritmo di slow-start

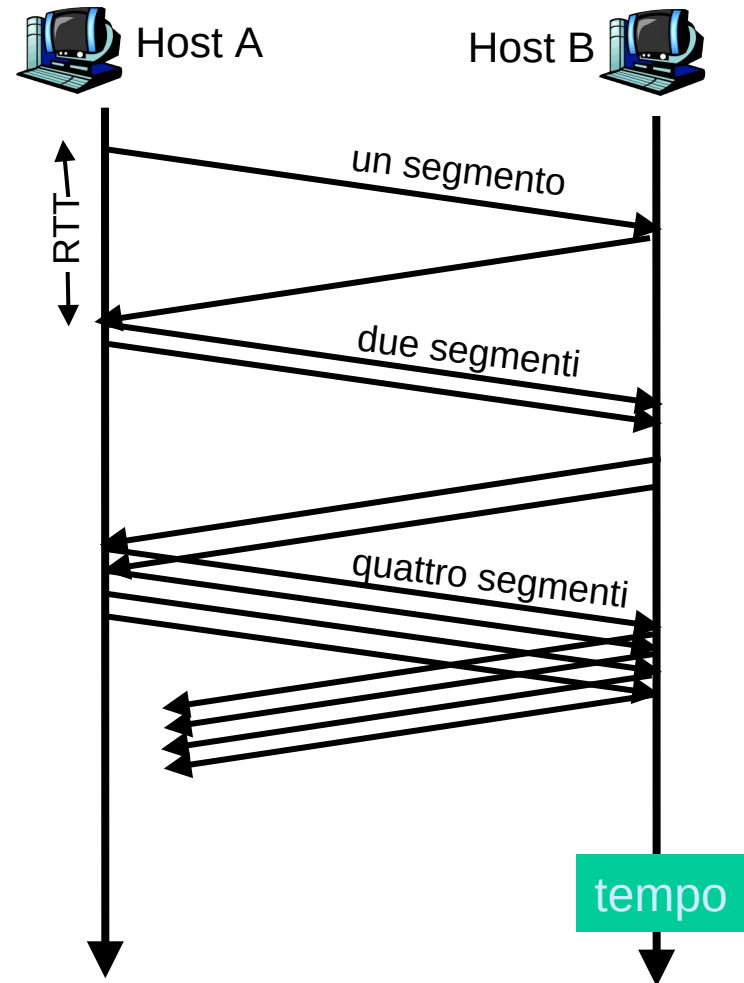
Inizializza: $cwnd = 1$
for (ogni segmento ACK)

$cwnd++$

until ((timeout OR
($cwnd \geq ssthresh$))

Aumento esponenziale → tempo di $\log_2(N) * RTT$ per arrivare a usare una finestra $cwnd$ di dimensione N

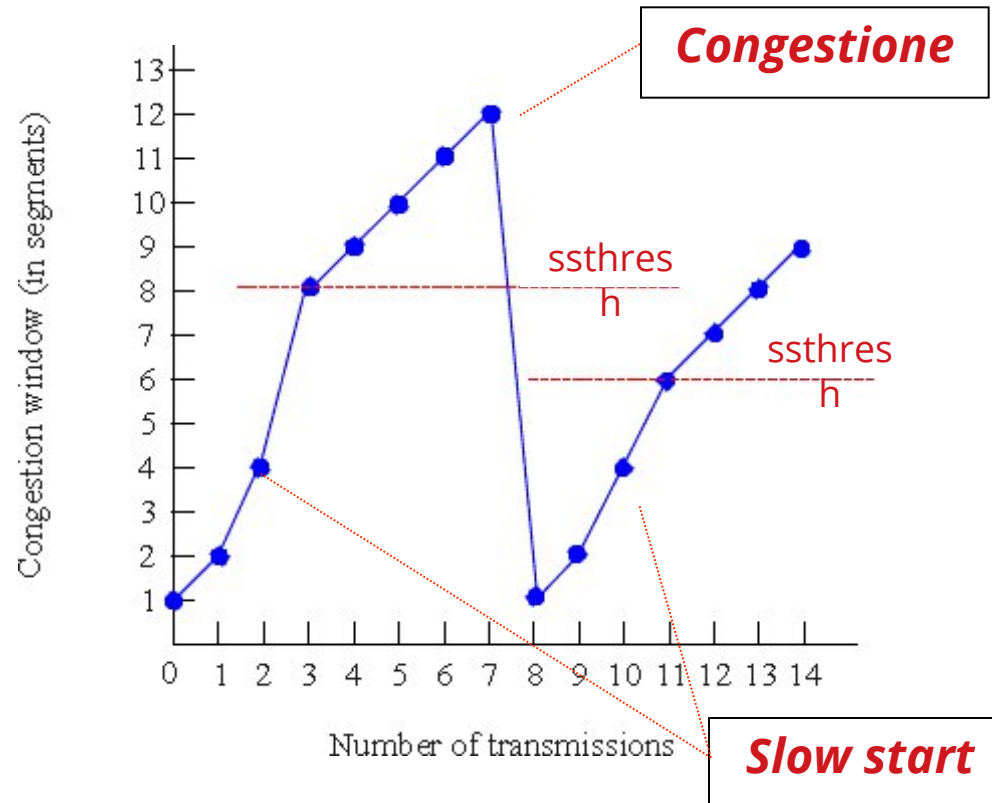
Nota: valore iniziale di $ssthresh$ tipicamente 65535 bytes (alcune implementazioni a infinito - 2^{31})



Congestion Avoidance

- Quando entra in CA, cwnd vale la metà di quello che valeva l'ultima volta che si è rilevata congestione (potenzialmente dietro l'angolo...)
 - **approccio più conservativo: incremento cwnd di 1 MSS ogni RTT**
- Incremento lineare
- TCP Tahoe (*prima versione*)

```
/* slow-start terminato */  
/* cwnd > ssthreshold */  
if not(perdita) {  
    ogni ACK ricevuto:  
    cwnd=cwnd+MSS(MSS/cwnd)  
}  
else { ssthreshold = cwnd/2;  
    cwnd = 1;  
    esegui slow-start }
```



Esempio incremento durante CA



- **MSS = 1460 byte**
- **cwnd = 14.600 byte (10 segmenti)**
 - **in un RTT vengono spediti 10 segmenti**
- **Ciascun ACK in arrivo (assumendo un ACK per segmento – no congestione) incrementa ampiezza finestra di 1/10 MSS**
 - **dopo ricezione di tutti 10 segmenti (in un RTT) cwnd sarà aumentato di 1 MSS**

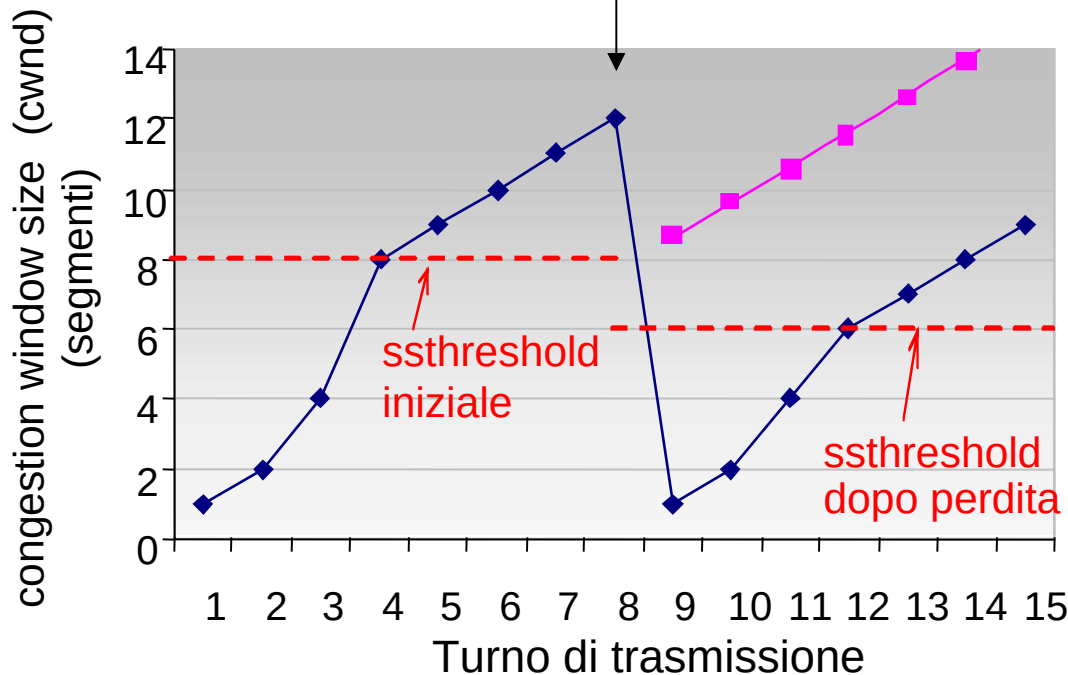
E nel caso di 3 ACK duplicati?



- **L'incremento lineare della CA termina quando si rileva di nuovo congestione**
 - Se si verifica un **timeout**, si riparte da **slow start**
 - $ssthresh = cwnd / 2$ e **$cwnd = 1 \text{ MSS}$**
 - **Cosa fare in caso di 3 ACK duplicati? Si può essere meno drastici?**
 - La rete sta ancora consegnando pacchetti – forse si è perso solo un pacchetto o era corrotto
 - Introduzione della **fase di fast recovery** nella versione più recente di TCP
 - **TCP Reno**
 - **In caso di 3 ACK duplicati**, non esagera riducendo $cwnd$ a 1, ma **dimezza la finestra della metà aggiungendo 1 MSS per ogni ACK duplicato ricevuto**: $\rightarrow cwnd = cwnd / 2 + 3 \text{ MSS}$
(e la $ssthresh$ è portata a $cwnd / 2$)
- e prosegue in CA**

TCP Reno vs. TCP Tahoe

Al turno 8 (cwnd = 12), si verifica una perdita di pacchetto rilevata da un triplo ACK duplicato



TAHOE:

- $ssthreshold = cwnd/2$
- $cwnd = 1$
- Slow start

RENO:

- $ssthreshold = cwnd/2$
- $cwnd = ssthreshold + 3MSS$
- CA (inc. lineare)

—◆— TCP Tahoe
—■— TCP Reno

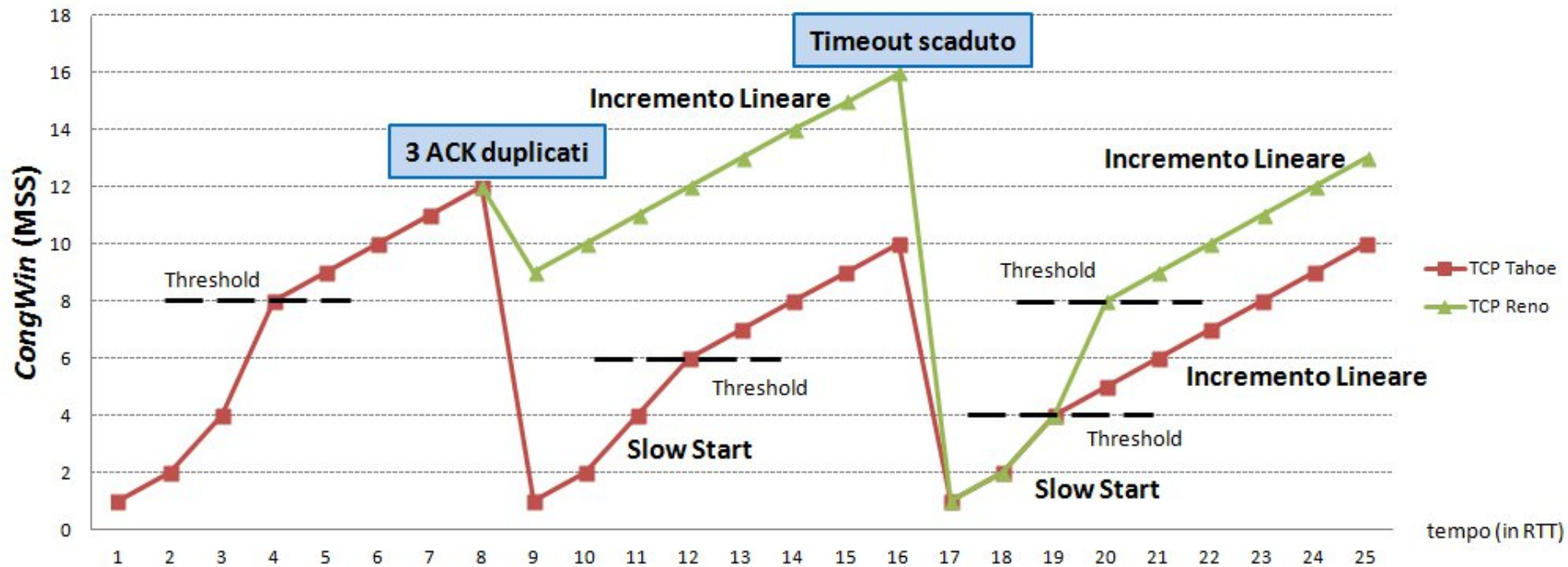
TCP Reno utilizza il *fast recovery*:
cwnd riprende da threshold + 3 e non da 1,
ma con crescita lineare e non esponenziale

TCP Reno vs. TCP Tahoe

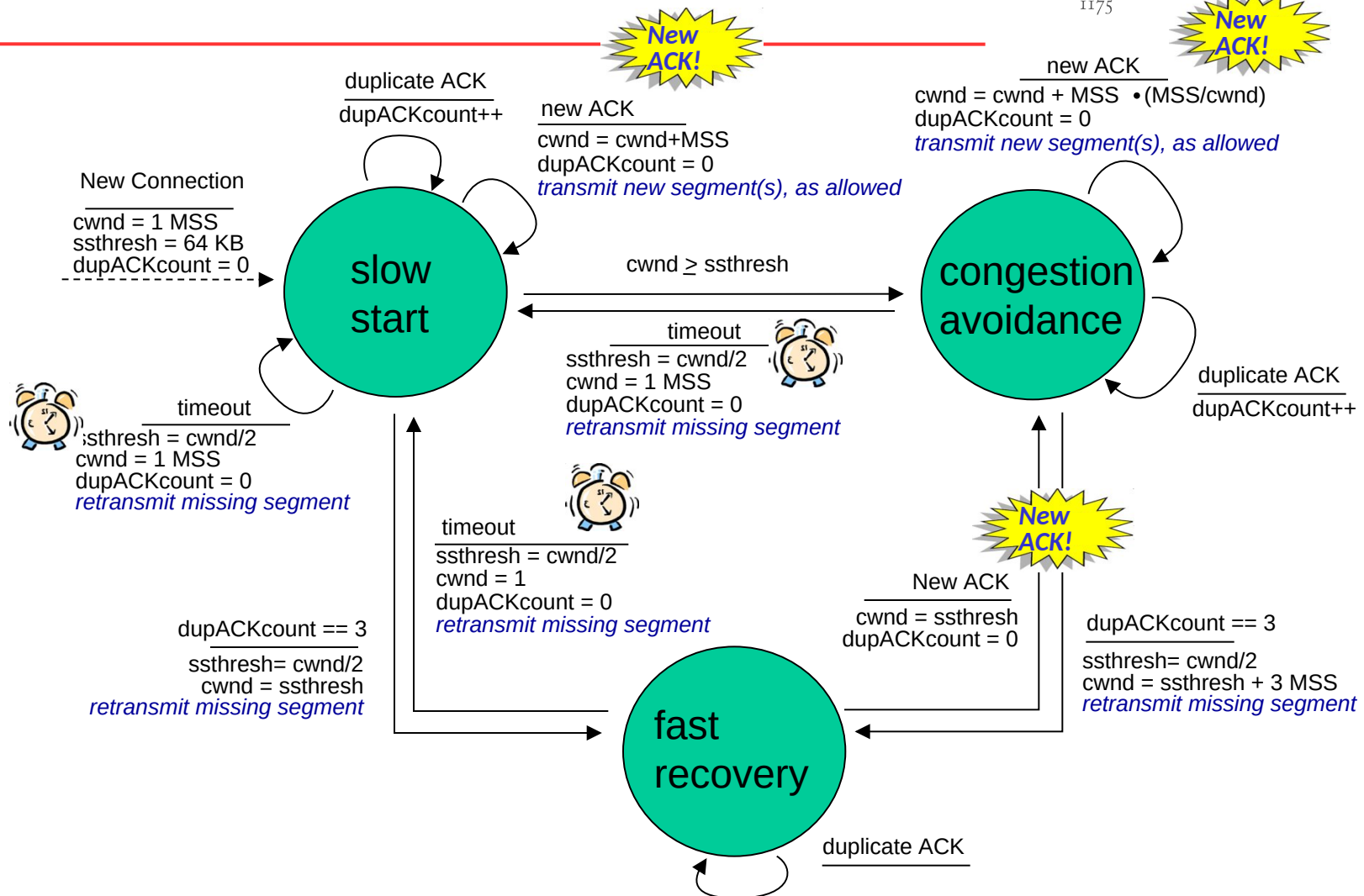


UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Evoluzione di cwnd in TCP Tahoe e TCP Reno



Riassumendo

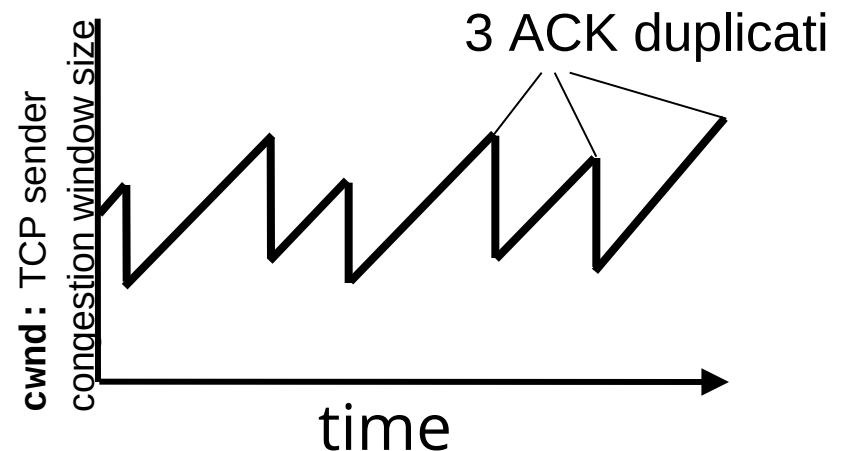


- **Visione complessiva**
- Tralasciando la fase di slow start, il controllo di congestione TCP Reno segue un **approccio AIMD per la congestion window**
 - **Additive Increase - Multiplicative Decrease**

Esempio

- Aumenta la window size linearmente per ACK arrivati entro timeout
- Diminuisce la window di un fattore moltiplicativo in caso di perdita rilevata con ACK duplicati (es., dividi la window size di 2)

AIMD = comportamento
a dente di sega

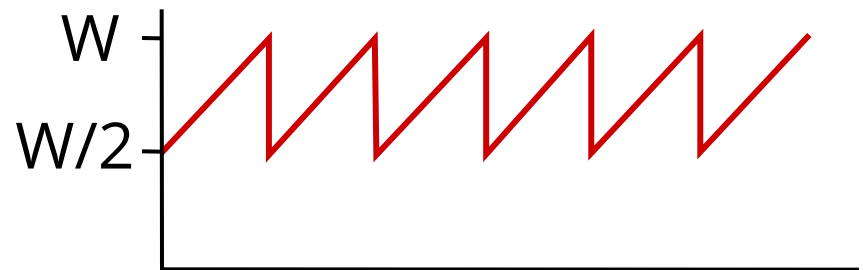


- **Maggior parte delle attuali implementazioni usa TCP Reno**
- Sono state proposte tante varianti. Es.,
- **Vegas**: cerca di predire congestione nei router intermedi osservando il valore del RTT
- **BIC, CUBIC**
- **Approccio AIMD di TCP Reno**: sviluppato sulla base di **esperienza tecnica sul campo e sperimentazioni**
- Dimostrato dopo anni di analisi teoriche che l'algoritmo di controllo di congestione TCP svolge la **funzione di un algoritmo distribuito di ottimizzazione asincrona** che ottimizza contemporaneamente **molteplici indici di prestazione** per la rete

TCP throughput



- **Quale potrebbe essere il throughput medio di una connessione TCP prolungata?**
- Ignoriamo fasi slow start (molto brevi, incremento esponenziale)
- Velocità di invio dei dati è funzione di cwnd e RTT corrente
- Se $cwnd = w$ byte, la frequenza trasmissiva è circa w/RTT
- TCP cerca banda aggiuntiva aumentando di 1 MSS a ogni RTT fino ad un evento di perdita
- Detto W il valore di w quando si verifica la perdita, e assumendo W e RTT circa costanti per la durata della connessione, la velocità trasmissiva varia tra $W/(2RTT)$ e W/RTT

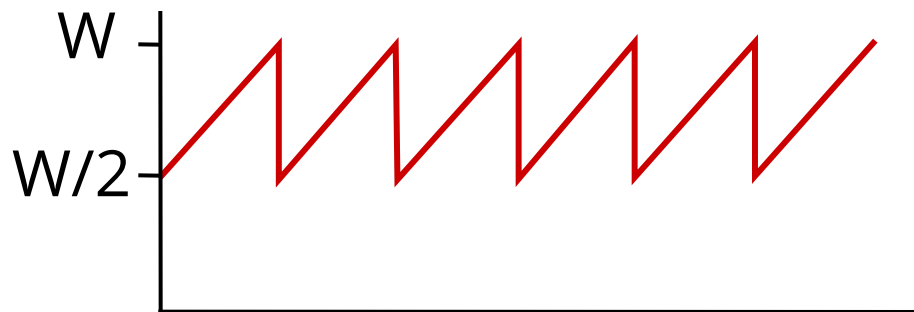


TCP throughput



- Modello macroscopico estremamente semplificato del comportamento a regime di TCP
- La dimensione media della congestion window $cwnd$ è $\frac{3}{4} W$
- Dato che il throughput cresce in maniera lineare tra i due estremi della finestra:

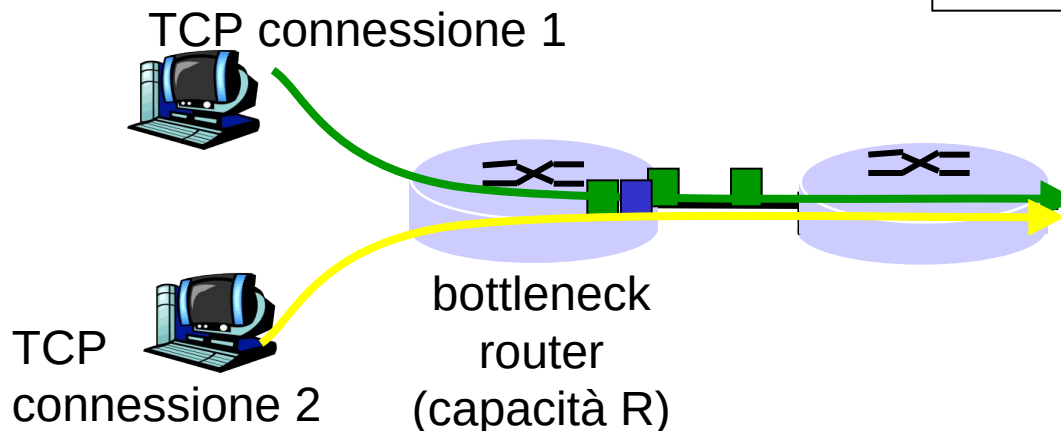
$$\text{TCP throughput} = \frac{3}{4} \frac{W}{RTT} \text{ bytes/sec}$$



Fairness TCP

- Consideriamo diverse sessioni TCP con percorsi end-to-end diversi ma che condividono un router che rappresenta il collo di bottiglia del sistema
- **Scopo della *fairness***: se vi sono N sessioni TCP che condividono lo stesso link, ciascuna deve ottenere $1/N$ -mo della capacità del link

***AIMD è un
approccio FAIR?***



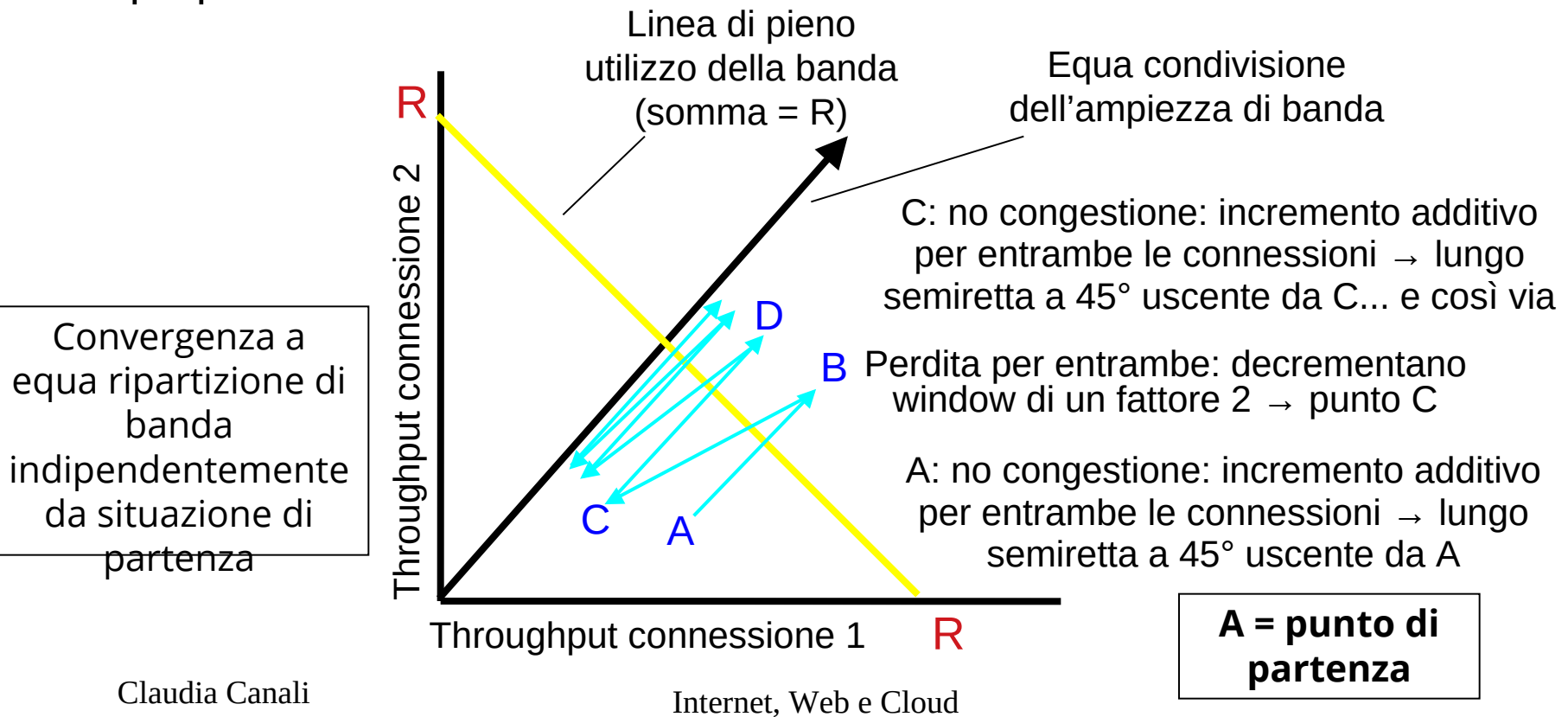
Le connessioni TCP possono avere inizio in istanti diversi → diverse dimensioni della finestra cwnd

Perchè TCP è fair?

Due sessioni in competizione su un link a capacità limitata R

- **Incremento additivo:** fa crescere il throughput
- **Decremento moltiplicativo:** diminuisce il throughput proporzionalmente

Assumiamo stessi
valori di RTT e MSS



Fairness e UDP



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

- Il controllo di congestione consente a TCP di **regolare il tasso trasmissivo** delle applicazioni
- Motivo per cui **applicazioni multimediali come fonia over IP e videoconferenza** non fanno uso di TCP: non vogliono che il flusso sia regolato anche se la rete è congestionata
- Usano UDP per immettere sulla rete dati a frequenza costante: occasionalmente meglio perderli che dover ridurre il tasso trasmissivo
- Dal punto di vista del TCP: le applicazioni UDP **non collaborano** – non son fair
- Dato che TCP abbassa il proprio tasso trasmissivo in risposta alla congestione e UDP no, le sorgenti UDP possono **soffocare il traffico TCP**
- **Area di ricerca attuale:** meccanismi di controllo congestione per evitare che traffico UDP blocchi il throughput di Internet

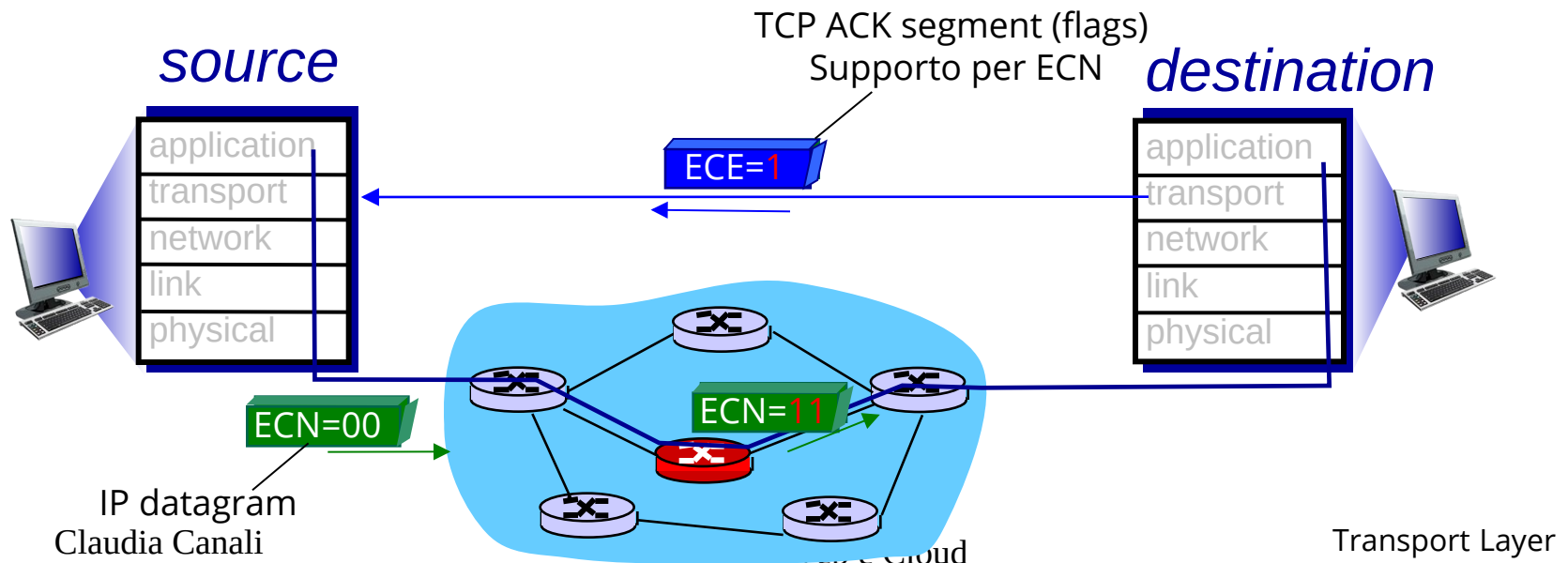
Explicit Congestion Notification (ECN)



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

- Controllo di congestione assistito dalla rete: estensioni di IP e TCP [RFC 3168]
- Segnalazione esplicita di congestione a livello rete
- Uso di 2 bit nell'intestazione del datagramma IP
- Router congestionato imposta i 2 bit a 11: l'informazione arriva al destinatario che avvisa esplicitamente il mittente attraverso un segmento di ACK
 - *Bit ECE (ECN-Echo) nei bit Not Used*
- Il mittente dimezza la cwnd come farebbe in caso di 3 ACK duplicati

Opzionale
In pratica non usato



→ E' vietata la copia e la riproduzione dei contenuti e immagini in qualsiasi forma.

E' inoltre vietata la redistribuzione e la pubblicazione dei contenuti e immagini non autorizzata espressamente dall'autore o dall'Università di Modena e Reggio Emilia."